Student Management System

Student Name: Amira Ashraf

Student ID: 12218

Project Overview

Console-Based Application

A C++ implementation demonstrating object-oriented programming principles through a comprehensive student record management system with persistent data storage capabilities.

Core Technologies

Standard Template Library (STL) containers including maps, vectors, and sets for efficient data management and retrieval operations.

Key Features

Student data management with add, remove, search, sort, and display operations along with course enrollment tracking and file-based persistence.

Project Objectives

Demonstrate OOP Concepts

Implementation of classes, inheritance, and encapsulation through Student and Person class hierarchy with proper member functions and data abstraction.

Master STL Containers

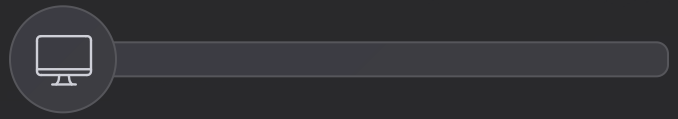
Utilization of map for ID-based student lookup, set for course management, and vector for sorting operations to showcase efficient data structure selection.

Implement File Persistence

Development of save and load functionality using fstream library to ensure data integrity across program sessions with proper error handling.

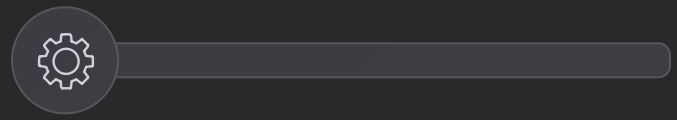
System Architecture

The Student Management System follows a layered architecture pattern that separates concerns between user interface, business logic, and data management layers for maintainability and scalability.



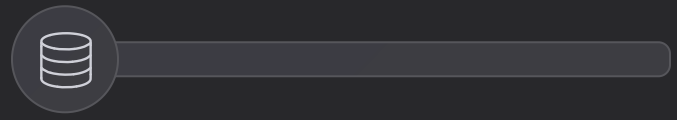
Presentation Layer

Menu-driven console interface providing intuitive user interaction through numbered command selection and formatted data display with clear prompts and error messages.



Logic Layer

Core functionality implemented in main.cpp with functions handling add, remove, search, sort, and enrollment operations while maintaining data consistency and business rules enforcement.



Data Layer

Student and Person classes managing data storage using STL containers (map) for O(log n) lookup performance and set for course uniqueness constraints.

Class Design Structure

Person Base Class

The foundational class providing common attributes and behaviors that can be extended by specialized student entities.

Attributes

- string name - Student full name storage

Methods

- Constructor for object initialization
- getName() accessor function

Student Derived Class

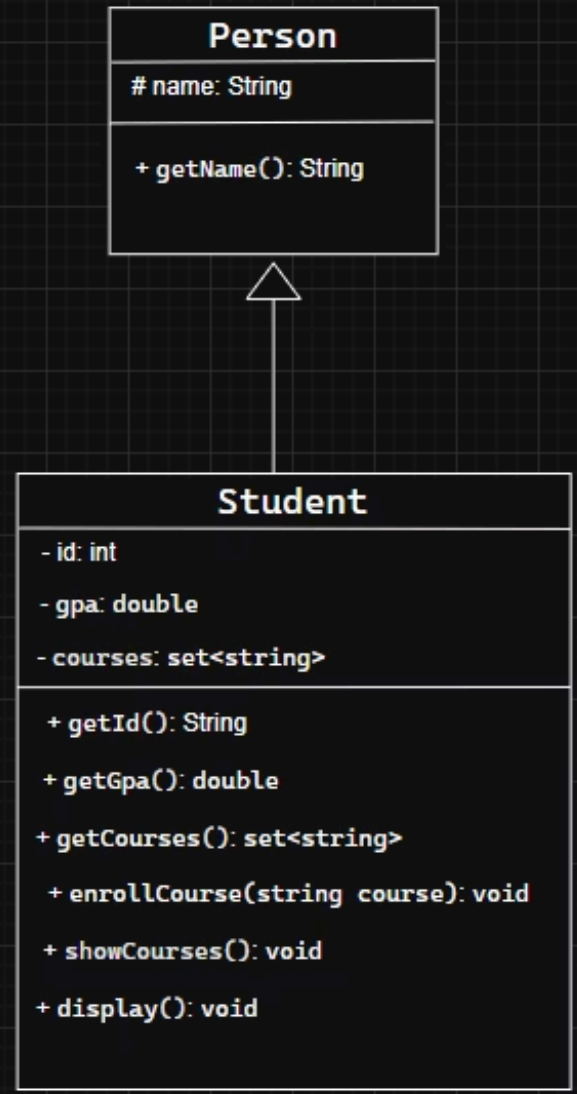
Extends Person class with additional academic attributes and enrollment management capabilities through inheritance.

Attributes

- int id - Unique student identifier
- double gpa - Grade point average tracking
- set courses - Enrolled courses storage

Methods

- enrollCourse() for course registration
- display() for formatted output
- showCourses() for enrollment listing



Core Features and Operations

1

Add Student

Insert new student records with unique ID validation and GPA range checking to maintain data integrity while preventing duplicate entries.

2

Remove Student

Delete existing student by ID with confirmation and automatic cleanup of associated course enrollments from the system.

3

Search Student

Quick lookup by ID or name using map's O(log n) search performance with formatted display of matching student details.

4

Display All

Show complete student roster with formatted output of all records including ID, name, GPA, and course enrollment status.

5

Enroll Course

Add courses to student record using set container to automatically prevent duplicate course entries with uniqueness constraint.

6

Sort Students

Arrange students by GPA using vector sorting algorithm for academic performance analysis and ranking visualization.

7

File Operations

Save current student data to disk and reload on program start for persistent storage across sessions with proper file I/O handling.

Development Timeline

The project was completed over a three-week development cycle with structured phases for design, implementation, testing, and documentation to ensure quality deliverables.

Week 1: Design Phase

Class design and UML diagram creation, data structure selection analysis, and feature requirement specification with test case planning.

1

Week 2: Implementation

2

Core functionality coding including class implementation, menu system development, file handling integration, and STL container integration.

Week 3: Testing

3

Test case execution with edge case validation, bug identification and fixing including file loading issue resolution, and final documentation.

Test Results Analysis

Successful Operations

- Student addition with duplicate ID detection
- Student removal by ID with validation
- Search functionality by ID and name
- Course enrollment with duplicate prevention
- Data saving to file with integrity checks
- Student display with formatted output

Areas for Improvement

- File loading implementation with range checking
- Enhanced error recovery mechanisms
- Input validation for edge cases
- Graceful handling of corrupted files
- Transaction rollback on failure

Conclusion and Future Directions

Project Achievements

The Student Management System successfully demonstrates fundamental C++ programming concepts including object-oriented design, STL container utilization, and file handling operations. The implementation showcases proper class hierarchy through inheritance, efficient data structure selection with map for fast lookups, and practical application of file I/O for data persistence.

Key Learnings

- STL container selection and performance characteristics
- File handling with proper error checking
- Menu-driven interface design patterns
- Object-oriented principles in practice

Future Enhancements

- Graphical user interface using Qt or Windows API
- Database integration with SQLite or MySQL
- Network capabilities for multi-user access
- Advanced reporting and analytics features